
U-Net per il denoising di file audio

June 22, 2025

Fabiana Manzella, 2075322 Sonia Darabi, 2077457 Sara Riccio, 2069698

Abstract

Il nostro progetto di Deep Learning si basa su un approccio di supervised-learning e si propone come obiettivo quello di costruire un modello in grado di ripulire dei file audio, appositamente modificati applicando del rumore Gaussiano. Dopodiché, confrontiamo i risultati ottenuti dal nostro modello con quelli di un modello built-in (il modello Demucs, di Facebook AI Research), in modo da capire quali aspetti migliorare in contesti di Deep Learning dove è utile saper utilizzare il denoising.

1. Introduzione

La scelta di questo topic è partita principalmente dal progetto ID02, che riguardava l'uso della libreria MusicGen, per generare brevi audio musicali da ripulire. Tuttavia, il nostro approccio è poi stato diverso, in quanto il dataset di suoni era inizialmente pulito. In seguito, abbiamo applicato del rumore gaussiano controllato, in modo tale da aiutare il modello a comprendere quale sia il rumore da eliminare. Abbiamo anche eseguito un'accurata analisi dei dati e una grid search personalizzata su alcuni parametri. Il modello che abbiamo scelto di adottare è una U-Net. In seguito, confrontiamo i valori ottenuti con quelli di un modello built-in utilizzato per il denoising di immagini.

2. Lavori correlati

Lo studio del denoising audio non è un lavoro poco oggetto di studio; al contrario, è molto attenzionato principalmente in ambito musicale e telefonico. Uno modello che si basa sul denoising audio è quello di Demucs (Deep Extractor for Music Sources) (Défossez et al., 2019), da cui abbiamo attinto per poter confrontare il denoising del nostro mod-

ello con uno built-in. Demucs è un modello pre-addestrato creato da Facebook AI Research (FAIR), che nasce principalmente per la separazione delle sorgenti audio, ma che in questo caso abbiamo utilizzato come denoiser, in quanto le sue abilità nel separare tracce audio possono renderlo efficace per rimuovere rumori in sottofondo. La differenza fra il nostro approccio e quello di Demucs consiste nel tipo di input dato al modello: mentre il nostro riceverà le trasformate di Fourier del segnale, Demucs lavora solamente sulle waveform audio. Possiede un'architettura UNet che è stata appunto adattata alle waveforms; si differenzia dalle semplici U-Net perché possiede un BLSTM che modella dipendenze a lungo raggio. Il modello, pertanto, riesce a comprendere le relazioni temporali globali, non solo quelle locali.

Un altro campo in cui trova spazio il denoising di immagini riguarda la medicina, in particolare al radiografia e le immagini. Il medical imaging risulta essere estremamente importante, in quanto spesso la qualità dell'immagine influisce direttamente sulle diagnosi e sui pareri medici. Di recente, sono stati proposti diversi approcci deep learning per il denoising medico, tra cui autoencoder e U-Net, che superano i metodi classici (come filtri mediani, Gaussiani o BM3D).

Uno dei primi studi che ha utilizzato il denoising è stato proposto da Chen et al. (Chen et al., 2017), che utilizzano un residual encoder-decoder CNN per ripulire immagini TAC a bassa dose. Il modello impara a predire il rumore e lo sottrae dal segnale rumoroso, mantenendo le strutture anatomiche rilevanti. Questo approccio è abbastanza simile a quello che abbiamo implementato nel nostro progetto, basato su residual learning con U-Net. Il denoising delle immagini è un aspetto centrale anche nella visione artificiale e nella guida autonoma dei veicoli, i cui sensori spesso vanno incontro a problematiche come scarsa illuminazione o condizioni atmosferiche poco favorevoli. Un esempio concreto dell'applicazione della U-Net in ambito di guida autonoma è il lavoro di Zhao et al. (Zhao et al., 2020), il cui modello (una U-Net) sfrutta denoising e segmentazione semantica per migliorare le prestazioni in condizioni di rumore elevato, che si presentano nelle dashcam notturne o in ambienti degradati.

Email: Fabiana Manzella
<manzella.2075322@studenti.uniroma1.it>, Sonia Darabi
<darabi.2077457@studenti.uniroma1.it>, Sara Riccio <riccio.2069698@studenti.uniroma1.it>.

Machine Learning 2025, Sapienza University of Rome, 2nd semester a.y. 2024/2025.

3. Metodologia

3.1. Dataset

Abbiamo importato il dataset FMA (Free Music Archive) con la versione small, contenente 8000 brani royalty-free di 30 secondi ciascuno, in modo da non avere problemi di import. Ogni traccia è completa e ben etichettata, in modo da esser certe di non avere problemi di tracce totalmente vuote o non complete. Dopo, abbiamo scaricato via script (wget, curl) in Colab direttamente l'archivio <https://github.com/mdeff/fma> Inizialmente è stato fatto sampling, scegliendo in maniera casuale 3000 tracce da utilizzare, raccogliendo tutto in una cartella sample/dataset, ma impostando un seed, in modo che i risultati potessero essere riproducibili. Abbiamo scelto il formato .wav per rappresentare le tracce musicali, poiché non comprime la traccia e in questo modo evitiamo delle perdite di informazioni. Abbiamo convertito quindi da .mp3 a .wav usando ffmpeg, usando anche il comando -ar 44100 che è il formato standard CD, e -ac 1 per impostare un solo canale audio. Dopodiché, abbiamo suddiviso il dataset in train, val e test (80, 10, 10), creando una sottocartella per ognuno. Ad ogni traccia è poi stata applicata la trasformata di Fourier tramite la libreria 'librosa' (McFee et al., 2022) (in particolare con la funzione librosa.stft) e per ogni traccia ne è stata creata una rumorosa corrispondente, tramite l'aggiunta di rumore gaussiano (con np.random.normal) alla trasformata. Il modello è infatti stato addestrato su un dataset con le trasformate di Fourier, non direttamente sui segnali audio. Dopodiché, abbiamo preso le trasformate pulite e quelle rovinare, salvandole in due array e convertendoli in tensori PyTorch X e Y, con una shape [N, 1, H, W], dove N è il numero degli esempi, 1 è il numero dei canali, H e W rappresentano invece le dimensioni delle trasformate di Fourier. Come ultima cosa, sono state implementate le classi Dataset e DataLoader.

3.2. Analisi dei dati

Abbiamo effettuato varie analisi dei dati, sia per mostrare le differenze tra l'audio pulito e il corrispondente audio rumoroso, sia per proprio far vedere come sono fatti gli audio. Abbiamo quindi utilizzato la libreria "librosa" (McFee et al., 2022) per ottenere le funzioni di queste analisi. Abbiamo estratto un file di audio dal dataset e su cui abbiamo visualizzato il grafico del segnale audio nel tempo, lo spettrogramma (la distribuzione delle frequenze nel tempo) e lo spettrogramma logaritmico, l'RMS (misura quanto è forte il segnale in media), la Banda Spettrale (misura quanto si estendono le frequenze in un determinato istante), il Centroide Spettrale (il "baricentro" dello spettro), il Flatness Spettrale (indica quanto il suono è simile al rumore bianco), gli MFCC (rappresentano il contenuto timbrico del suono) e lo Zero Crossing Rate (Conta quante volte il segnale at-

traversa lo zero). Infine, abbiamo anche calcolato la correlazione tra audio pulito e rovinato (per vedere quanto i due segnali fossero simili) e la correlazione media su tutto il dataset del training, della validation e del test, insieme all'SNR stimato (misura quanto il segnale utile è più forte del rumore) e l'SNR medio di nuovo su tutti i tre sottoinsiemi del dataset.

3.3. Definizione del modello

Il modello che abbiamo scelto di implementare autonomamente è una U-Net, chiamata 'ImprovedUnet2', per distinguerla dal modello built-in che testeremo in seguito. Abbiamo quindi una rete neurale convoluzionale (CNN), che abbiamo scelto appositamente per l'analisi di immagini. E' stata costruita in PyTorch con 3 sezioni principali: un encoder, che riduce progressivamente la dimensione dello spettrogramma in analisi aumentando il numero di feature maps e facendo downsampling, tramite MaxPool2d(2); un bottleneck, come snodo fra encoder e decoder; un decoder, co cui si esegue un upsampling progressivamente, combinando poi le feature fra encoder e decoder con le skip connections. Si passa poi il tutto ad una funzione forward, la quale ritorna un output denoised. In seguito, viene messo il modello sulla GPU, nel caso fosse disponibile, altrimenti nella CPU. Infine, l'ottimizzatore scelto è Adam.

3.3.1. Loss

La funzione di perdita definita è una loss personalizzata, la *Spectral Loss*. Questa combina l'errore quadratico medio (MSE) sia in un dominio lineare che in uno logaritmico, con la seguente formula:

$$\mathcal{L}_{\text{spectral}} = \alpha \cdot \text{MSE}(\hat{y}, y) + (1 - \alpha) \cdot \text{MSE}(\log(\hat{y}), \log(y))$$

dove x è il segnale predetto, y il target e α bilancia le due componenti. La grid search è stata implementata anche su α , in modo tale da trovare il miglior valore possibile fra [1, 0.7, 0.5, 0]. Nella ricerca degli iperparametri è emerso che il modello performa meglio quando $\alpha = 1$, ovvero quando la loss utilizzata è la MSE lineare.

3.4. Training

Durante il training è stata poi implementata una grid search per la ricerca degli iperparametri che dessero i migliori risultati in termini di loss e validation loss: la ricerca ha impiegato al programma tempi variabili per ogni combinazione. In particolare, la ricerca degli iperparametri ha riguardato batch size e il numero di epoche, il learning rate per l'ottimizzatore Adam e il parametro α . E' presente anche un grafico che mostra i grafici di train loss e validation loss. Qui sotto un'immagine della grid search compiuta sugli iperparametri 'batch size = 4, 8; lr = 1e-4, 1e-5' con gli iperparametri fissi 'epochs = 5: alpha = 1'.

Il grafico della discesa della loss è stato fatto in base agli iperparametri 'lr = 1e-4, batch size = 8'. Con questi iperparametri è stato poi addestrato il modello per 20 epoche.

```
Testing batch_size=4, epochs=5, lr=0.0001, alpha=1, (Adam)
epoch 1/5 - Train: 0.0283 - Val: 0.0174
epoch 2/5 - Train: 0.0121 - Val: 0.0157
epoch 3/5 - Train: 0.0160 - Val: 0.0152
epoch 4/5 - Train: 0.0154 - Val: 0.0147
epoch 5/5 - Train: 0.0150 - Val: 0.0148

Testing batch_size=4, epochs=5, lr=1e-05, alpha=1, (Adam)
epoch 1/5 - Train: 0.0178 - Val: 0.0222
epoch 2/5 - Train: 0.0211 - Val: 0.0199
epoch 3/5 - Train: 0.0195 - Val: 0.0193
epoch 4/5 - Train: 0.0185 - Val: 0.0178
epoch 5/5 - Train: 0.0179 - Val: 0.0188

Testing batch_size=8, epochs=5, lr=0.0001, alpha=1, (Adam)
epoch 1/5 - Train: 0.0233 - Val: 0.0168
epoch 2/5 - Train: 0.0188 - Val: 0.0157
epoch 3/5 - Train: 0.0160 - Val: 0.0155
epoch 4/5 - Train: 0.0156 - Val: 0.0158
epoch 5/5 - Train: 0.0153 - Val: 0.0148

Testing batch_size=8, epochs=5, lr=1e-05, alpha=1, (Adam)
epoch 1/5 - Train: 0.0603 - Val: 0.0123
epoch 2/5 - Train: 0.0256 - Val: 0.0206
epoch 3/5 - Train: 0.0287 - Val: 0.0198
epoch 4/5 - Train: 0.0197 - Val: 0.0191
epoch 5/5 - Train: 0.0190 - Val: 0.0182
```

Figure 1. Andamento di train loss e validation loss con 4 combinazioni di iperparametri, ognuna su 5 epoche.

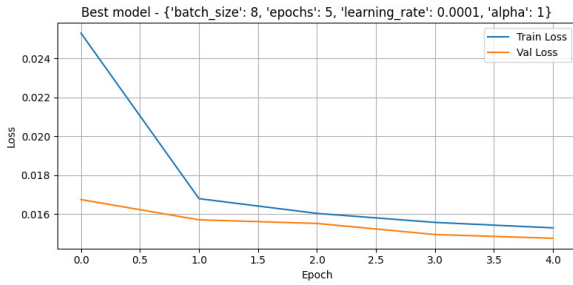


Figure 2. Grafico di train loss e validation loss con 5 epoche.

3.5. Testing

Nella funzione di testing, dopo aver fatto nuovamente pre-processing sui dati, il modello viene valutato anche con alcune metriche molto comuni per la valutazione del denoising. Abbiamo inserito PSNR (Peak Signal-to-Noise Ratio), che misura il rapporto tra la potenza del segnale originale e la potenza dell'errore (rumore) introdotto dalla ricostruzione, con la seguente formula:

$$PSNR = 10 \cdot \log_{10} \left(\frac{MAX^2}{MSE} \right)$$

Un valore ottimale per la PSNR è superiore ai 20.0 dB, altrimenti un valore inferiore, misura, oltre che una riduzione del rumore, anche una perdita di informazione. Un'altra metrica spesso usata è SSIM (Structural Similarity Index) non misura solo l'errore pixel a pixel, ma anche quanto la struttura visiva di due immagini è simile, imitando la percezione umana, con la seguente:

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}$$

In questo caso, un buon valore si aggira intorno a valori vicini a 1, più precisamente inferiori a 1.

Inoltre, abbiamo inserito degli spettrogrammi, in modo tale da visualizzare concretamente la differenza fra l'audio 'pulito' e quello a cui è stato applicato rumore.

3.6. Metriche di valutazione

Abbiamo utilizzato nuovamente le funzioni precedentemente usate per le analisi, per mostrare la differenza tra input rumoroso, output del modello e la versione pulita dell'audio. Inoltre abbiamo definito 3 altre funzioni per valutare il modello:

- FAD (Fréchet Audio Distance), che confronta la distribuzione degli embedding tra ground truth (audio pulito) e l'audio restituito dal modello, valutando quanto il dominio delle feature dell'audio generato si discosta da quello dell'audio reale.
- STOI (Short-Time Objective Intelligibility), che valuta la chiarezza dopo il denoising, restituendo un punteggio tra 0 e 1.
- OpenL3 Embeddings Similarity, che genera una rappresentazione (embedding) del contenuto audio e calcola la similarità tra le versioni pulite e quelle restituite dal modello.

4. Risultati sperimentali

Dato che i modelli convoluzionali vengono solitamente applicati a dataset di immagini, sono stati fatti diversi tentativi sulla preparazione di un dataset che fosse adatto per essere utilizzato nell'addestramento della U-Net sui file audio. Inizialmente abbiamo provato ad aggiungere il rumore gaussiano direttamente ai segnali audio, per poi convertirli in spettrogrammi e creare un dataset con ground truth corrispondente agli spettrogrammi degli audio puliti e input corrispondente agli spettrogrammi degli audio sporchi, da passare al modello. Un altro tentativo è stato di calcolare dapprima gli spettrogrammi, per poi inserire su di essi il rumore gaussiano, e passarli al modello. Entrambi gli approcci hanno prodotto risultati scarsi perché nel passaggio da segnale a spettrogramma si perdono delle informazioni, in particolare è stato difficile ricostruire i segnali audio dopo il test per valutare l'efficacia del denoising. In entrambi i casi abbiamo calcolato gli spettrogrammi a partire dal segnale con le funzioni di librosa: $S = np.abs(librosa.stft(y, n_fft=1024, hop_length=512))$ $S_db = librosa.amplitude_to_db(S, ref=np.max)$. Ci siamo accorte che in particolare il calcolo del valore assoluto rende l'operazione non facilmente reversibile. Abbiamo allora pensato di applicare il rumore ad un passaggio intermedio, la trasformata di Fourier del segnale ($librosa.stft(y, n$

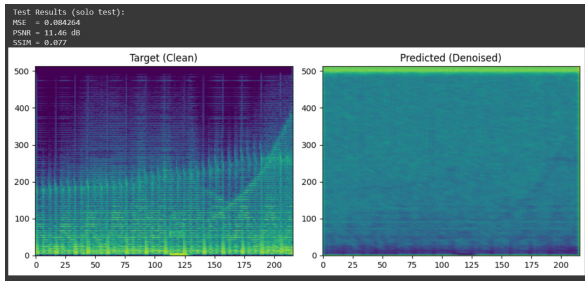


Figure 3. Spettrogrammi di una traccia audio 'pulita' e poi 'ripulita' tramite il modello precedente.

fft=1024, hop length=512)) e abbiamo ottenuto un notevole miglioramento nella qualità dei risultati e nelle metriche. I risultati del modello ImprovedUNet, inizialmente, sembravano poco promettenti: l'utilizzo della semplice MSE risultava poco efficiente, anche se il grafico che descriveva il comportamento di train loss e validation loss sembrava indicare che il modello stesse apprendendo molto. Pertanto, è stato definito un secondo modello (ImprovedUNet2), con delle migliorie:

- **Residual Learning:** con quest'aggiunta, il modello riesce a prevedere il rumore da sottrarre all'input, non a costruire l'immagine ripulita; in particolare, risulta utile in quanto il rumore ha una struttura più 'semplice' da individuare rispetto all'immagine intera.
- **SEB (Squeeze and Excitation Block):** è un blocco che aiuta il modello a porre più attenzione ai canali più rilevanti e ad ignorare quelli meno utili; è stato implementato alla fine di ogni blocco convoluzionale
- **Dropout2d e LeakyReLU:** Dropout2d è una versione specializzata per immagini, che non azzerava solamente dei neuroni, ma degli interi canali; LeakyReLU, invece, è una variante della funzione di attivazione ReLU, che prova ad evitare il problema dei neuroni 'morti', ovvero quelli che sono 0 o molto vicino allo 0 con ReLU.
- **Final Layer:** il final layer di ImprovedUNet2 possiede 2 strati, invece che uno, in questo modo una struttura più profonda migliora la non-linearità del mapping finale.

Ecco alcuni dei risultati migliori che abbiamo ottenuto, con un PSNR = 2.17 dB e un SSIM = 0.016:

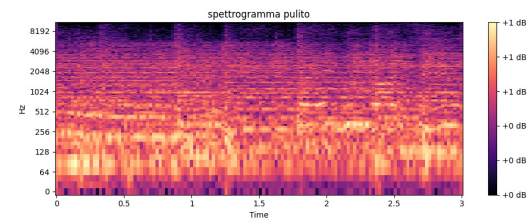


Figure 4. Spettrogramma di una traccia audio 'pulita'.

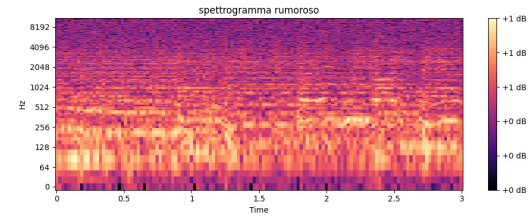


Figure 5. Spettrogramma di una traccia audio 'rumorosa'.

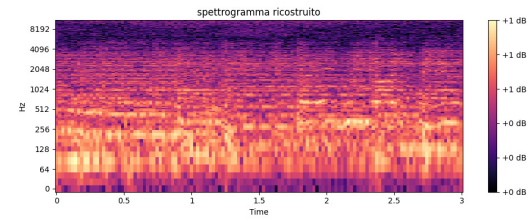


Figure 6. Spettrogramma di una traccia audio 'ripulita'.

Un esperimento che abbiamo effettuato poi, è stato quello di prelevare una traccia audio dal dataset fma small e di testarla con Demucs, che nonostante non sia un modello specificatamente creato per il denoising audio, ma per la separazione delle sorgenti, ha comunque prodotto dei buoni risultati, con un PSNR = 30.87 dB e un SSIM = 0.8202.

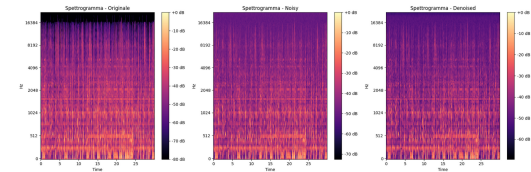


Figure 7. Spettrogrammi di Demucs.

5. Discussione e conclusione

Crediamo che i risultati ottenuti possano essere abbastanza promettenti; un limite molto importante è stato imposto dalla nostra indisponibilità di una GPU attiva in qualsiasi momento, ma anche dalla scarsa RAM di cui pote-

vamo usufruire. Probabilmente, con un numero maggiore di epoche e disponendo di una maggiore quantità di dati con cui fare training, i risultati sarebbero stati ancora migliori. Per motivi organizzativi, oltre ai compiti specificati tramite mail, il modello è stato sviluppato da Manzella Fabiana (2075322), il debugging e il training da Riccio Sara (2069698), Darabi Sonia (2077457) si è occupata della valutazione del modello e del debugging.

Bibliography.

- Google. *VGGish: A pretrained model for audio classification*. Disponibile su: <https://tfhub.dev/google/vggish/1>.
- OpenL3 Developers. *OpenL3 Audio Embedding Tutorial*. Disponibile su: <https://openl3.readthedocs.io/en/latest/tutorial.html#using-the-library>.
- NumPy Developers. *numpy.corrcoef — NumPy Documentation v2.2*. Disponibile su: <https://numpy.org/doc/2.2/reference/generated/numpy.corrcoef.html>.
- PyTorch Developers. *torch.nn.MSELoss — PyTorch Documentation*. Disponibile su: <https://pytorch.org/docs/stable/generated/torch.nn.MSELoss.html>.
- Takaki, S., Nakashika, T., Wang, X., Yamagishi, J. *STFT Spectral Loss for Training a Neural Speech Waveform Model*. Disponibile su: <https://arxiv.org/abs/1810.11945>.
- Radkoff, E. *Loss Functions in Audio ML – Sounds and Words*. Disponibile su: <https://www.soundsandwords.io/audio-loss-functions/>.
- Hu, J., Shen, L., Albanie, S., Sun, G., Wu, E. *Squeeze-and-Excitation Networks*. Disponibile su: <https://arxiv.org/abs/1709.01507>.
- Zhang, K., Zuo, W., Chen, Y., Meng, D., Zhang, L. *Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising*. Disponibile su: <https://arxiv.org/abs/1608.03981>.
- PyTorch Developers. *torch.nn.LeakyReLU — PyTorch Documentation*. Disponibile su: <https://pytorch.org/docs/stable/generated/torch.nn.LeakyReLU.html>.
- Minh Tran. *Understanding U-Net*. Disponibile su: <https://towardsdatascience.com/understanding-u-net-61276b10f360/>.

- PyTorch Developers. *torch.nn.Dropout2d — PyTorch Documentation*. Disponibile su: <https://docs.pytorch.org/docs/stable/generated/torch.nn.Dropout2d.html>.
- Simon J.D. Prince *Understanding Deep Learning*.
- Stewart Russell and Peter Norvig *Artificial Intelligence A Modern Approach*.

References

- Chen, H., Zhang, Y., Zhang, W., Liao, P., Li, K., Zhou, J., and Wang, G. Low-dose ct with a residual encoder-decoder convolutional neural network. *IEEE Transactions on Medical Imaging*, 36(12):2524–2535, 2017.
- Défossiez, A., Usunier, N., Bottou, L., and Bach, F. Music source separation in the waveform domain. *arXiv preprint arXiv:1911.13254*, 2019.
- McFee, B. et al. Librosa documentation v0.11.0. <https://librosa.org/doc/0.11.0/>, 2022.
- Zhao, S., Liu, Y., Zhang, W., and Yang, L. A multi-task u-net for joint denoising and road scene segmentation. In *2020 IEEE 23rd International Conference on Intelligent Transportation Systems (ITSC)*, pp. 1–6. IEEE, 2020.